

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1408 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 10%

Duration :60 Mins

On Class

QUESTIONS: 1 Total Marks:50

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

I Vishal B(192524485) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Vishal.B



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lexical Analysis and Token Specification using LEX

Presented by:

Vishal B(192524485)

Guided by:

Dr.G.Michael
Department of CSE

Introduction

- Lexical analysis is the first phase of a compiler.
- It reads source code character by character.
- It groups characters into meaningful tokens.
- It removes white spaces and comments.
- It reports illegal characters as errors.

Phases of a Compiler

- Lexical Analysis – Converts characters into tokens.
- Syntax Analysis – Checks grammar.
- Semantic Analysis – Checks meaning.
- Intermediate Code Generation.
- Code Optimization.
- Code Generation.

Tokens, Lexemes and Patterns

- Token – A category name for a group of similar words. (e.g. keyword, identifier)
- Lexeme – The actual sequence of characters matched. (e.g. "total")
- Pattern – The rule that describes a token's form. (e.g. [a-zA-Z]+)

Token Specification

- Identifier: `[a-zA-Z_][a-zA-Z0-9_]*`
- Number: `[0-9]+`
- Keyword: `if` | `else` | `while`
- Operator: `+` `-` `*` `/`
- White spaces are ignored.

Types of Tokens

- Keywords – if, else, while, int
- Identifiers – names given by the programmer
- Constants – numbers and fixed values
- Operators – +, -, *, /, =
- Punctuation – ; , () { }
- Literals – "text" and characters

LEX – Introduction

- LEX is a lexical analyzer generator.
- Developed for Unix systems.
- Uses regular expressions.
- Produces C source code for scanners.
- Works with YACC.

Structure of LEX Program

- Definition Section – Header files and declarations.
- Rules Section – Regular expressions and actions.
- User Code Section – Additional C functions.
- Sections are separated by %% symbols.

Working of LEX

Source Program



LEX Scanner



Pattern Matching



Token Generation



Parser (YACC)



Syntax Tree

Advantages and Applications

Advantages:

- Fast token generation
- Easy maintenance
- Reduces manual coding

Applications:

- Compilers
- Interpreters
- IDEs
- Syntax highlighting

Conclusion

- LEX simplifies lexical analysis.
- It improves compiler efficiency.
- It is widely used with parser generators like YACC.

